The Core Workflows

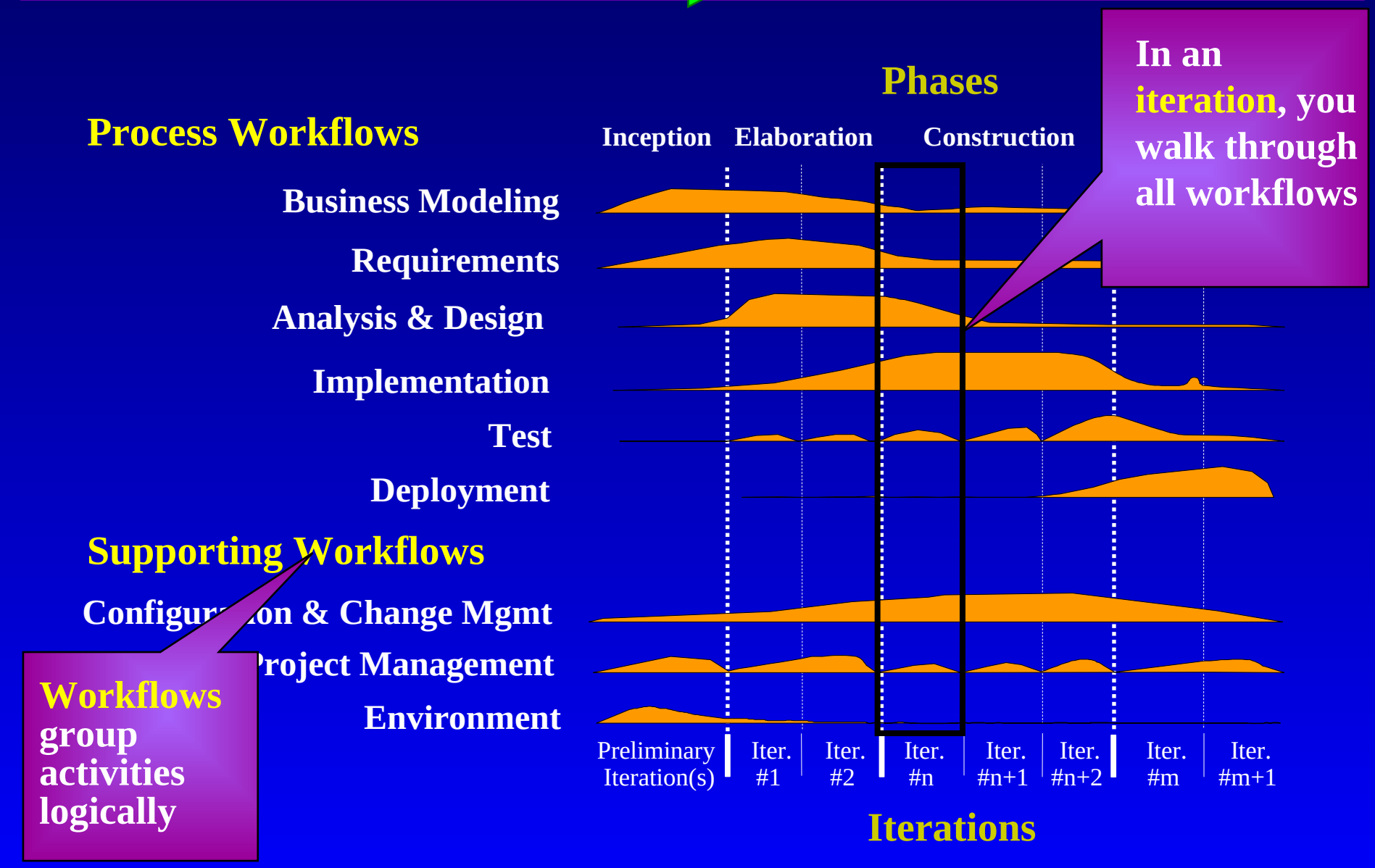
Requirements Capture:

From Vision to Requirements

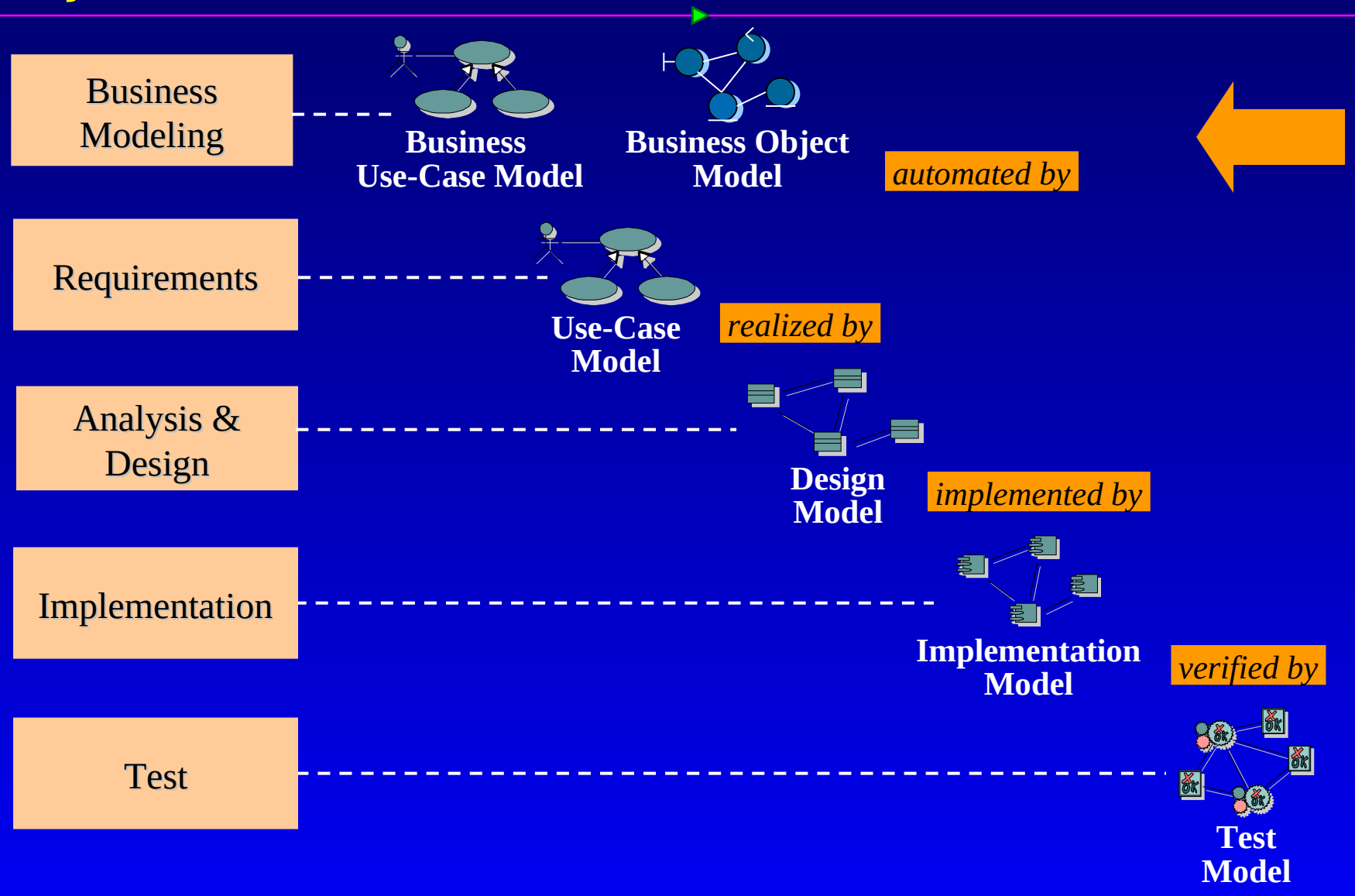
The Core Workflows

- Requirements
  - requirements capture — Chapter 6
  - requirements as use cases — Chapter 7
- Analysis — Chapter 8
- Design — Chapter 9
- Implementation — Chapter 10
- Test — Chapter 11

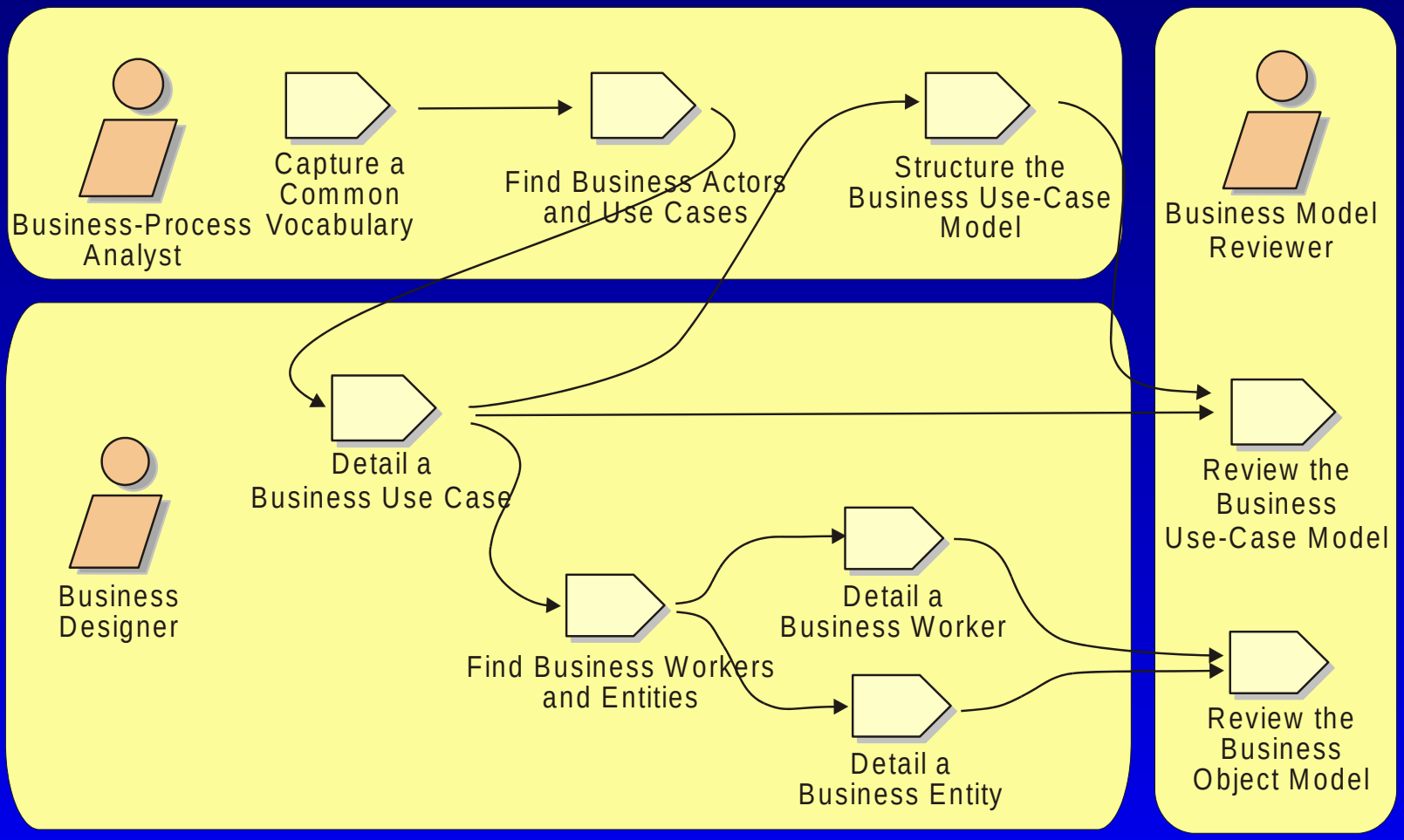
Recall: The Iterative Model from RUP



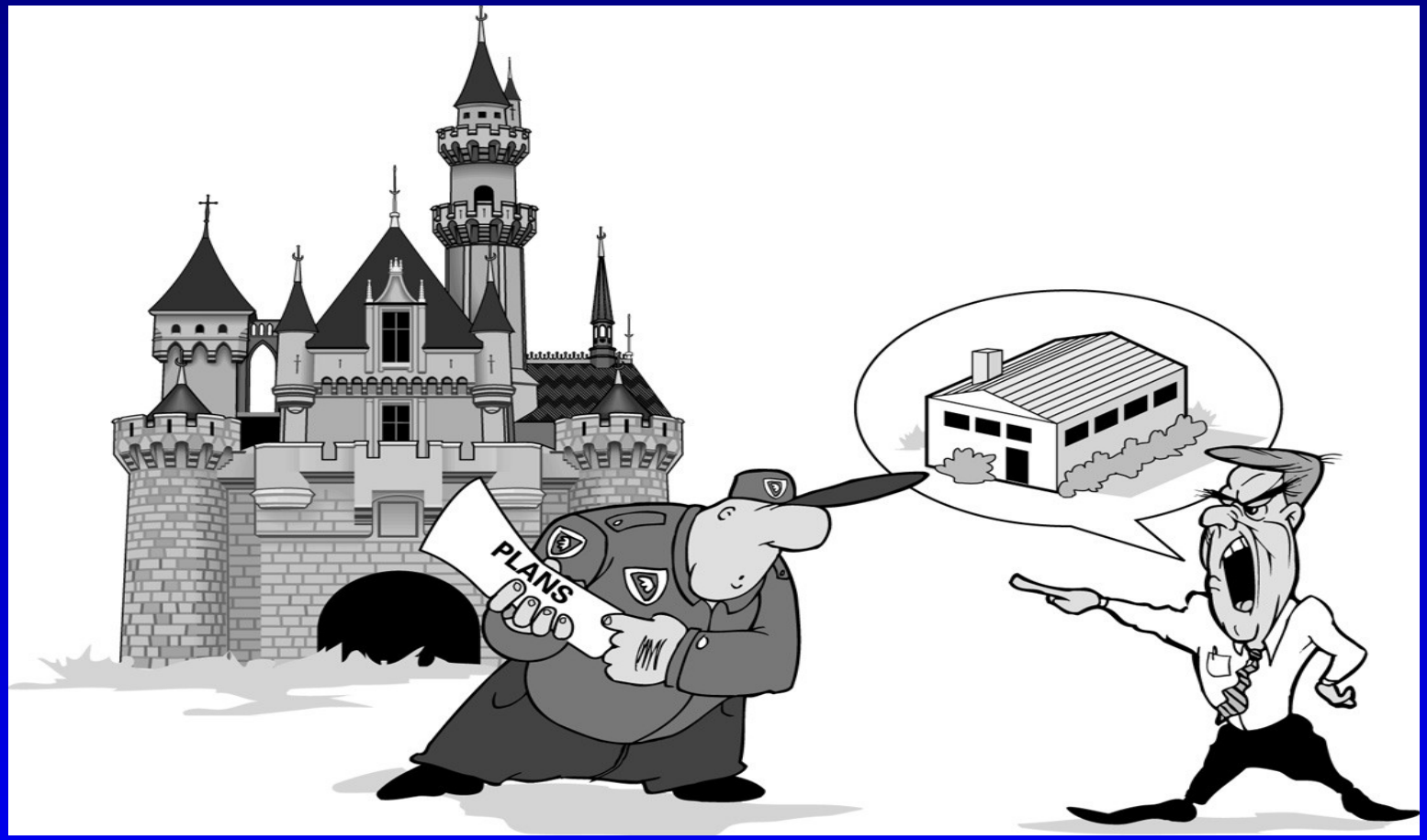
Major Workflows Produce Models



Business Modeling Workflow



Purpose of Requirements Capture



Purpose of Requirements Capture

- To achieve agreement on what is to be produced.
- To decrease ambiguity and increase consistency and completeness.
- To do this, understand the customer need.
  - If you do not understand what the customer wants you will fail
  - Seek first to understand and then to be understood [Covey]
- To document the agreed set of requirements.
  - It is not enough just to understand what the customer wants, it is also necessary to record that understanding

SHARED VISION

Purpose of Requirements Capture

- To identify key issues: requirements with strong influence on cost, schedule, functionality, risk or performance
- To provide a basis for system design
- To provide a reference point for system validation
- What if there are problems in the customer provided system specification?

COMMUNICATION

Why Requirements Capture is Difficult

- Sophisticated problem solving required
- Impossible to capture user needs completely
- Desire to truncate the activity and “progress” to the next activity
  - schedule pressure
- Customers failure to cooperate in verifying requirements are correct
- Assumptions are made which are not documented or discussed
- “How” instead of “What” is written
- Engineers lack of training in requirements engineering
- We don’t pay attention to lessons learned

Overview of Requirements Capture

- Initial requirements often need elaboration
- Requirements elicitation and elaboration sources:
  - Operational Concepts and Scenarios
  - System Specifications
  - Interviews/questionnaires
  - Observation of the operational environment
  - Use cases
  - Project Champion
  - Prototyping poorly understood requirements

Overview of Requirements Capture

- Often customers have only a vague notion what they want
- General strategy for requirements capture
  - list candidate requirements
  - understand system context
  - capture functional requirements
  - capture nonfunctional requirements

List Candidate Requirements

- Create feature list
- For each feature
  - short name and description
  - status (proposed, approved, incorporated, validated)
  - estimated cost to implement (person-hours)
  - priority (must, should, could)
  - risk to implement (ordinary, significant, critical)
- Use list to allocate requirements to implementation iterations

Understand System Context

- Domain modeling
  - describe important concepts of the application as domain objects
  - link objects to one another
  - gives glossary of terms
  - some classes may make it into the design
- Business modeling
  - describe existing (or perceived) process
  - establish workers, responsibilities and operations performed
  - helps in later capturing use cases

Capture Functional Requirements

- Use cases can be used to capture functional requirements
  - represent one way of using the system
  - each user needs several use cases, typically
- Should also capture user interface which is exercised for each use case
  - sketch several versions
  - discuss with customer
- Do not get upset if customers change what you show them
  - better now than later

Capture Nonfunctional Requirements

- Performance
- Platform dependencies
- Quality
  - maintainability
  - extensibility
  - reliability
  - availability
  - usability
  - …
- Design and implementation constraints

Resulting Artifacts

Work to be done	Resulting Artifacts
List candidate requirements	Feature list
Understand system context	Business or domain model
Capture functional requirements	Use-case model
Capture nonfunctional requirements	Supplementary requirements or individual use cases

System Context Using a Domain Model

- Captures most important types of objects in the system
- Many come from the requirements specification or interviews of experts
- Typical kinds of objects or classes
  - business objects, such as orders, projects, accounts
  - real-world objects needing to be tracked, such as inventory
  - events that may happen, such as end-of-month
- Described in class diagrams

Developing a Domain Model

- Workshops by domain analysts
- Describe the *most important* classes in the domain
  - 10-50 classes in a medium domain
- Less important classes just listed in glossary for later use
- For very small systems (hint hint) an object model for the domain may be overkill
  - use a glossary of terms instead
- Purpose is to understand the context with a view of understanding problem

Where is the Domain Model Used?

- Helps to establish vocabulary for later phases
- Helps to describe the use cases and design the user interface
- Helps to suggest classes which will come up in analysis
- Developing a business model even better for these
  - domain model is special case of business model

System Context Using a Business Model

- Used for understanding the business processes of an organization
- Identify the use cases and relevant business entities
  - model just enough to understand the context
- Supported by two UML models
  - use-case models
  - object models

What is a Business Model?

- Business use-case model
  - business use cases and business actors
  - presents business from the usage perspective
  - much of what is modeled may be outside system being developed
- Business object model
  - describes how each use case is realized by workers using business *entities* and *work units*
    - *entity* is something which workers access, inspect manipulate, produce or use in a business use case
    - *work unit* is a set of entities which form a recognizable whole to an end user

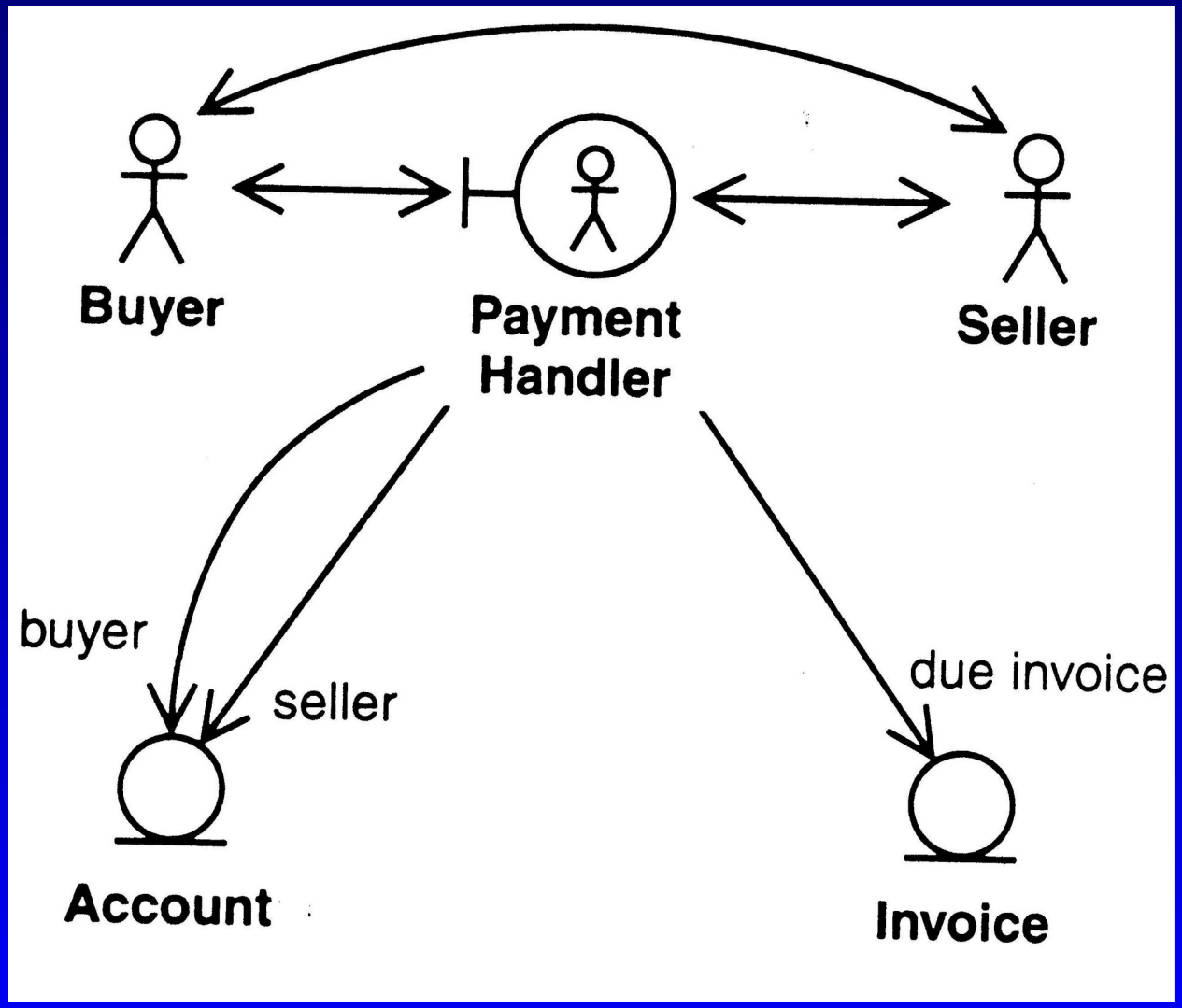
Example - The Sales: From Order to Delivery Case

Workers take the following steps in the business use case Sales: From Order to Delivery

1. A buyer orders goods or services by contacting the seller
2. The seller sends an invoice to the buyer through the payment handler
3. The seller delivers the goods or services to the buyer
4. The buyer pays via the payment handler. This involves transferring money from the account of the buyer to the account of the seller.

The payment handler is a bank employee who helps in steps 2 and 4. These tasks could be automated by an information system

Sales: From Order to Delivery



How to Develop a Business Model

- Prepare a business use case model
  - identify actors
  - identify business use cases used by the actors
- Develop a business object model
  - workers
  - business entities
  - work units
  - as required by use cases
- Business object model captures the things the workers need to work with
  - entities similar to domain classes in domain model

Summary

- Business and Domain models help define system context
- Use cases are used to capture the requirements
- Involves customer experts, application experts, methodology experts
- Builds the basis for project understanding
- Builds the basis for project scope